

Laboratorium 6: Kompresja danych – proste metody / Kodowanie Huffmana

Opracowanie zestawu zadań (część programistyczna)

Kacper Bednarczuk

Kod programu

Program został napisany w środowisku Node.js. Implementuje algorytm RLE ze znacznikiem, operując na buforach bajtów, co pozwala na bezstratną kompresję dowolnych plików binarnych (w tym grafik).

```
// Wbudowany moduł do obsługi systemu plików
const fs = require("fs");

// Definicja znacznika - używamy '$' (kod ASCII 36)
// W prawdziwych systemach szukalibyśmy bajtu, który nie występuje w pliku
const MARKER = 36;

function encode_rle(data) {
  // Konwersja wejścia na Buffer, jeśli jeszcze nim nie jest
  const buffer = Buffer.isBuffer(data) ? data : Buffer.from(data);
  const result = [];
  let i = 0;

  // Pętla przetwarzająca dane bajt po bajcie
  while (i < buffer.length) {
    let count = 1;
    // Zliczanie powtórzeń (max 255, bo licznik musi zmieścić się w 1 bajcie)
    while (
      i + count < buffer.length &&
      buffer[i] === buffer[i + count] &&
      count < 255
    ) {
      count++;
    }

    // Reguła 1: Seria 3 lub więcej znaków -> [ZNACZNIK, LICZNIK, WARTOŚĆ]
    // LUB Reguła bezpieczeństwa: Jeśli znak to sam ZNACZNIK, musimy go
    zakodować jako serię
    if (count >= 3 || buffer[i] === MARKER) {
```

```

        result.push(MARKER);
        result.push(count);
        result.push(buffer[i]);
    }
    // Reguła 2: Literał (mniej niż 3 powtórzenia i nie jest to znacznik)
    else {
        for (let k = 0; k < count; k++) {
            result.push(buffer[i]);
        }
    }

    // Przesunięcie indeksu o liczbę przetworzonych znaków
    i += count;
}

// Zwracamy wynik jako Buffer
return Buffer.from(result);
}

function decode_rle(encoded_data) {
    const buffer = Buffer.isBuffer(encoded_data)
        ? encoded_data
        : Buffer.from(encoded_data);
    const result = [];
    let i = 0;

    // Iteracja po zakodowanych danych
    while (i < buffer.length) {
        // Jeśli napotkamy znacznik, odczytujemy serię
        if (buffer[i] === MARKER) {
            // Pobranie licznika (następny bajt)
            const count = buffer[i + 1];
            // Pobranie wartości (kolejny bajt)
            const value = buffer[i + 2];

            // Odtworzenie serii znaków
            for (let k = 0; k < count; k++) {
                result.push(value);
            }
            // Przeskakujemy 3 bajty (Znacznik + Licznik + Wartość)
            i += 3;
        }
        // W przeciwnym razie traktujemy jako literał
        else {
            result.push(buffer[i]);
            i++;
        }
    }

    return Buffer.from(result);
}

```

```

}

function analyse_compression(name, original, compressed) {
  console.log(`\n=== Analiza zbioru: ${name} ===`);
  console.log(`Rozmiar oryginalny:  ${original.length * 8} bitów`);
  console.log(`Rozmiar po RLE:      ${compressed.length * 8} bitów`);

  const ratio = (1 - compressed.length / original.length) * 100;
  // Wyświetlanie wyniku z dokładnością do 2 miejsc po przecinku
  console.log(`Stopień kompresji:  ${ratio.toFixed(2)}%`);

  if (compressed.length < original.length) {
    console.log(`Status:          SUKCES (Kompresja)`);
  } else if (compressed.length > original.length) {
    console.log(`Status:          EKSPANSJA (Rozrost danych)`);
  } else {
    console.log(`Status:          BRAK ZMIAN`);
  }
}

// --- TESTY (Zbiory z części teoretycznej) ---

// 1. Zbiór A (Tekst)
const setA = "BABA_JAGA_BABA";
const encodedA = encode_rle(setA);
const decodedA = decode_rle(encodedA);

analyse_compression("Zbiór A (Tekst)", Buffer.from(setA), encodedA);
console.log(`Poprawność dekodowania: ${decodedA.toString() === setA}`);

// 2. Zbiór B (Grafika/Sekwencje)
const setB = "AAAAAAAAABBBBBBBBBBBBBBBBCCCCC";
const encodedB = encode_rle(setB);
const decodedB = decode_rle(encodedB);

analyse_compression("Zbiór B (Grafika)", Buffer.from(setB), encodedB);
console.log(`Poprawność dekodowania: ${decodedB.toString() === setB}`);

// --- TEST NA PLIKU GRAFICZNYM ---
const path = require("path");
const fileName = "image_8bit.bmp";
const filePath = path.join(__dirname, fileName);

try {
  if (fs.existsSync(filePath)) {
    console.log(`\n=== Test na pliku: ${fileName} ===`);

    // Odczyt pliku jako binarny Buffer
    const fileData = fs.readFileSync(filePath);
  }
}

```

```
// Kodowanie
console.time("Czas kodowania");
const encodedFile = encode_rle(fileData);
console.timeEnd("Czas kodowania");

// Dekodowanie (dla sprawdzenia spójności)
console.time("Czas dekodowania");
const decodedFile = decode_rle(encodedFile);
console.timeEnd("Czas dekodowania");

// Analiza wyników
analyse_compression("Bitmapa 8-bit", fileData, encodedFile);

// Sprawdzenie czy plik po dekompresji jest identyczny co do bajtu
const isValid = fileData.equals(decodedFile);
console.log(`Spójność danych (Hash check): ${isValid ? "OK" : "BŁĄD"}`);
} else {
  console.log(
    '\n[INFO] Brak pliku ${fileName} w katalogu roboczym. Pomięto test plikowy.'
  );
}
} catch (error) {
  console.error("Wystąpił błąd podczas przetwarzania pliku:",
error.message);
}
```

Testowane zdjęcie oraz jego 8-bitowa bitmapa

Do testu wykorzystano plik w formacie `.bmp`, ponieważ jest on formatem bezstratnym i nieskompresowanym, co pozwala na rzetelną ocenę efektywności algorytmu RLE. Format `.jpg` zawierałby już skompresowane dane, co mogłoby doprowadzić do ekspansji.

Oryginalne zdjęcie:



8-bitowa bitmapa (użyta w teście):



Wyniki testów i wnioski

Poniżej przedstawiono logi wygenerowane przez program:

```
=== Analiza zbioru: Zbiór A (Tekst) ===
```

```
Rozmiar oryginalny: 112 bitów
```

```
Rozmiar po RLE: 112 bitów
```

```
Stopień kompresji: 0.00%
```

```
Status: BRAK ZMIAN
```

```
Poprawność dekodowania: true
```

```
=== Analiza zbioru: Zbiór B (Grafika) ===
```

```
Rozmiar oryginalny: 256 bitów
```

```
Rozmiar po RLE:      72 bitów
Stopień kompresji:   71.88%
Status:              SUKCES (Kompresja)
Poprawność dekodowania: true
```

```
=== Test na pliku: image_8bit.bmp ===
```

```
Czas kodowania: 95.244ms
Czas dekodowania: 81.088ms
```

```
=== Analiza zbioru: Bitmapa 8-bit ===
```

```
Rozmiar oryginalny: 44433360 bitów
Rozmiar po RLE:      33454520 bitów
Stopień kompresji:   24.71%
Status:              SUKCES (Kompresja)
Spójność danych (Hash check): OK
```

Wnioski: Wyniki uzyskane z programu w pełni pokrywają się z obliczeniami teoretycznymi dla Zbiorów A i B. Dodatkowy test na rzeczywistej bitmapie wykazał kompresję na poziomie ok. 25%, co potwierdza użyteczność metody RLE dla prostych danych graficznych zawierających jednolite obszary kolorów.